arXiv:2512.02337v2 [cs.LG] 29 Aug 2026

SpecPV: Improving Self-Speculative Decoding for Long-Context Generation via Partial Verification

Zhendong Tan, Xingjun Zhang, Chaoyi Hu, Junjie Peng, Kun Xia
School of Computer Science and Technology, Xi'an Jiaotong University
772316639@stu.xjtu.edu.cn, xjzhang@xjtu.edu.cn

Abstract

Growing demands from tasks like code generation, deep reasoning, and long-document understanding have made long-context generation a crucial capability for large language models (LLMs). Speculative decoding is one of the most direct and effective approaches for accelerating generation. It follows a draft–verify paradigm, where a lightweight draft model proposes several candidate tokens and the target model verifies them. However, we find that as the context length grows, verification stage gradually becomes the dominant bottleneck. To further accelerate speculative decoding in long-context scenarios, we introduce **SpecPV**, a self-speculative decoding approach that performs fast verification using partial key–value states (KV) and periodically applies full verification to eliminate accumulated errors. We validate SpecPV across multiple long-context benchmarks and models, including LLaMA-3.1-8B-Instruct and Qwen3-series. Experimental results show that SpecPV achieves up to 6× decoding speedup over standard autoregressive decoding with negligible degradation. Our code is available at: https://github.com/TanZhendong/SpecPV.

1 Introduction

Long-context generation has become an essential capability for large language models (LLMs). Most modern models, such as DeepSeek and Qwen3 (Liu et al., 2024a; Yang et al., 2025a), now support context windows more than 128K tokens. The demand for long-context generation arises in two main scenarios. On the one hand, as LLMs become more powerful, applications such as document analysis, summarization, and code generation require processing much longer inputs, driving the need for extended context lengths. On the other hand, as think modes become more prevalent (Guo et al., 2025; Jaech et al., 2024), models are required to explicitly generate their reasoning process. This often leads to longer and even verbose outputs, which further amplifies the challenge of efficient long-context generation.

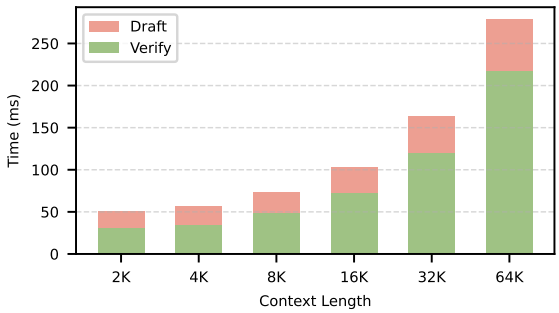

Figure 1: Drafting and verification time of EAGLE-3 speculative decoding on LLaMA-3.1-8B-Instruct. As the context length increases, verification gradually becomes the dominant bottleneck.

Many approaches have been proposed to improve the efficiency of long-context model generation, such as sparse attention (Tang et al., 2024; Xiao et al., 2024), KV cache quantization (Liu et al., 2024c), and even the exploration of new architectures (Wei et al., 2025; Yang et al., 2024). Beyond these, one of the most direct and effective methods is *speculative decoding*. During autoregressive generation, the model can only predict one token at a time, requiring repeated forward passes when producing long sequences. Speculative decoding alleviates this limitation through a *draft–verify* framework: a lightweight draft model first generates multiple candidate tokens, which are then verified by the target LLM in a single forward pass. Since the draft model is much faster and multiple tokens can be accepted at once, this approach substantially reduces decoding latency.

Recent studies have explored applying speculative decoding to long-context generation in pursuit of lossless acceleration (Sun et al., 2024; Wu et al., 2025; Sadhukhan et al., 2025). When handling long contexts, the primary challenges lie in the rapidly

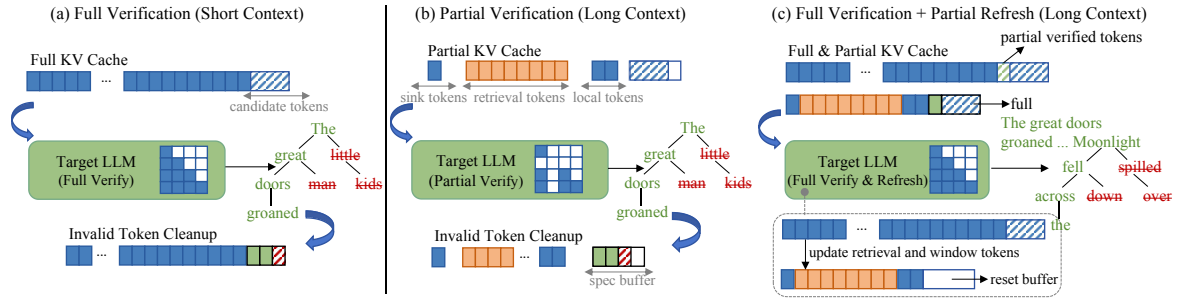

Figure 2: Illustration of three verification processes in SpecPV. For short context, we adopt classic full verification, whereas for long context, we use partial verification to improve efficiency. Periodic full verification eliminates accumulated errors and refreshes the partial KV cache. Taken together, these modes balance efficiency and accuracy across different context length.

growing KV cache and the complexity of attention. Consequently, these methods typically focus on designing draft models with lightweight KV cache, while relying on full KV during verification to ensure lossless.

However, when applying EAGLE-3 (Li et al., 2025b) to long-context generation, we measure the average drafting and verification time, as shown in Figure 1. For shorter contexts, although each draft step is relatively fast, searching and generating a tree-structured set of candidate tokens still incurs a non-negligible cost. As the context length increases, however, the verification time grows from around 60% to nearly 80%, gradually becoming the dominant bottleneck. This observation raises a fundamental question: *is full verification truly necessary in long-context speculative decoding, or can reliable verification be achieved using only partial KV states, thereby substantially reducing verification overhead?*

A natural concern arises that partial verification might harm model performance. However, numerous studies (Zhang et al., 2023; Tang et al., 2024; Xu et al., 2025) have shown that attention in LLMs exhibits strong sparsity, especially when processing long context. This indicates that, by employing carefully designed retrieval strategies, one can achieve effective verification using partial KV cache with negligible accuracy loss.

Building on this insight, we propose a partial verification approach for self-speculative decoding. In this framework, the target LLM and the lightweight draft module alternately perform forward passes. During verification, the target LLM obtains the layer features of accepted tokens, which are reused to guide subsequent drafts. Considering that LLMs inevitably accumulate errors during partial cache generation, we replace only part of the verification steps with partial verification, while periodically performing full verification to rectify deviations and ensure that the generated results remain consistent with the original outputs. Moreover, since the partial KV cache is much smaller than the full one, under memory-constrained settings we can offload only the full KV cache while keeping the partial cache on-device, thereby achieving even better speedup. Finally, we evaluate the model performance and efficiency of SpecPV across different models and datasets using the EAGLE-3 draft (Li et al., 2025b). Experimental results demonstrate that SpecPV incurs only negligible accuracy loss and provides substantial efficiency gains, achieving up to 6× decoding speedup compared to standard autoregressive decoding and around 2× speedup over full verification at a 60K context length.

In summary, our key contributions are as follows:

- We summarize the framework of Self-Speculative Decoding and identify its bottlenecks in long-context generation.
- We propose SpecPV, a speculative decoding framework with partial verification, which further accelerates long-context generation.
- We evaluate SpecPV across different models and datasets, showing up to 6× decoding speedup with negligible accuracy loss.

2 Related Work

Speculative Decoding Speculative decoding is a widely used approach for losslessly accelerating autoregressive models. Leviathan et al. (2023) introduced Speculative Sampling and demonstrated that it is theoretically equivalent to standard sampling.

A key challenge in speculative decoding is how to organize candidate tokens to improve the accepted length during drafting. Classical approaches employ a small speculative model to generate a single candidate sequence. SpecInfer (Miao et al., 2024) organizes candidate tokens into a tree, effectively increasing the number of accepted tokens by expanding the candidate space. Building on this, subsequent works (Li et al., 2024a; Liu et al., 2024b) have explored dynamic tree construction based on the draft model's predicted distribution.

Another key challenge lies in designing a fast yet effective draft model, as its speed and accept length directly determine the overall acceleration. Traditional approaches typically employ an independent small model, such as LLaMA-68M, to serve as the draft (Miao et al., 2024). Several studies leverage part of the target LLM's layer features to construct the draft model, a paradigm commonly referred to as self-speculative decoding. Specifically, some studies implement this paradigm through layer skipping or early exiting (Zhang et al., 2024; Elhoushi et al., 2024; Liu et al., 2024b; Xia et al., 2025). In contrast, methods such as EAGLE (Li et al., 2024b), Medusa (Cai et al., 2024), and Multi-Token Prediction (MTP) (Gloeckle et al., 2024) directly reuse the top-layer features as part of the draft model's input. Furthermore, EAGLE-3 (Li et al., 2025b) adopts a training-time test technique to avoid relying solely on the top layer and shifts from feature prediction to direct token prediction. Overall, self-speculative decoding enables more efficient draft prediction with less training data and smaller model size. Therefore, our work focuses on this paradigm to further enhance decoding efficiency in long-context generation.

Efficient Long-Context Generation The main bottlenecks in long-context generation lie in the rapidly growing KV cache and the complexity of attention. To address these challenges, several research directions have emerged. KV cache quantization (Liu et al., 2024c; Hooper et al., 2024) reduces memory usage and accelerates attention through faster data loading. Sparse attention (Xiao et al., 2024; Tang et al., 2024; Zhang et al., 2023) leverages the inherent sparsity of attention patterns, allowing models to compute attention over only a small subset of key–value pairs with minimal accuracy loss. New architectures, such as Linear Attention and Hybrid Linear Attention (Kimi Team, 2025), have also been actively explored as alternatives to full attention with superior efficiency.

Triforce (Sun et al., 2024) introduces streaming cache drafting and proposes a hierarchical speculation to alleviate memory bottlenecks. MagicDec (Sadhukhan et al., 2025) leverages a draft model with sparse KV cache to accelerate generation for large batch size. TokenSwift (Wu et al., 2025) employs partial KV cache and a Medusa-style draft model to achieve lossless acceleration for ultra long sequence generation. The aforementioned methods primarily focus on the draft phase, whereas our work targets the verification phase, adopting an EAGLE-3–style draft model and partial verification to further accelerate long-context generation.

3 SpecPV

In this section, we introduce SpecPV, a partial verification self-speculative decoding approach for long-context generation. We first introduce the inference framework of self-speculative decoding and identify its bottlenecks. Then, we present our partial verification method. Finally, to minimize potential performance degradation, we periodically adopt full verification to mitigate accumulated errors. The following subsections describe each component in detail.

3.1 Self-Speculative Decoding Framework

Traditional approaches typically employ a standalone small speculative model that shares the target LLM's vocabulary. In practice, this model is usually a smaller member of the same model family. In contrast, the key characteristic of self-speculative decoding is that the draft model reuses layer features from the target LLM. We provide an example in Figure 3 to better illustrate it.

After the prefilling stage, the model predicts the next token and generates a set of layer features that serve as inputs for self-speculative drafting. Different methods make use of these features in distinct ways: some rely solely on top-layer features, while others combine token representations with layer features. These methods also differ in how they produce multiple future tokens. Multi-head designs assign each head to a specific future position, whereas single-head approaches repeatedly apply one prediction head in an autoregressive manner. The draft module itself is typically lightweight, often consisting of only a single decoder layer.

A common misunderstanding is that drafting requires running the target LLM to obtain layer fea-

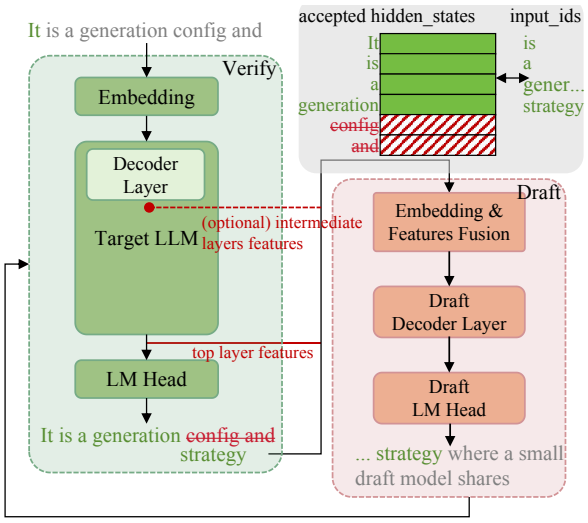

Figure 3: Illustration of the generation process in self-speculative decoding. A key characteristic of self-speculation is that the draft model reuses the layer features from the target LLM.

tures, making it appear as though the target model must participate in the draft phase. In practice, verification already produces all necessary layer features for the accepted tokens. The features are naturally aligned with the next step predictions, in which the feature corresponding to token x_i is used to draft token x_{i+1}. As a result, the draft phase consists solely of the draft module computation, without invoking any additional forward passes of the target LLM.

In long-context scenarios, most prior works primarily explore alternative drafting strategies while still relying on full verification to guarantee lossless acceleration. However, as shown in Section 1, we observe that as the context length grows, the verification stage progressively becomes the dominant performance bottleneck. This motivates our work to focus on accelerating the verification stage, aiming to substantially improve the efficiency of speculative decoding in long-context generation.

3.2 Partial Verification

As illustrated in Figure 2, we adopt partial verification to improve the decoding efficiency. Since current KV cache managers commonly organize the cache in block formats (Kwon et al., 2023), we also select partial KV cache at the block level.

The partial KV cache consists of four components. First, due to the *attention sink* mechanism (Xiao et al., 2024), keeping the KV blocks of the initial tokens will largely preserve the performance of window attention. Therefore, we always retain the first few KV cache blocks as *sink tokens*. Next is the *retrieval tokens*, which forms the main body of the partial KV cache. Inspired by Quest (Tang et al., 2024), the set of critical tokens depends on the current query. Owing to the semantic continuity of natural language, which endows attention with intrinsic locality (Yang et al., 2025b), the partial KV cache does not need to be updated at every step while still maintaining strong model performance. To further accelerate the retrieval process, we cache the key-states summary of each block.

$$C_i = \left(K_i^{\max},\ K_i^{\min}\right) \tag{1}$$

where $K_i^{\max}$ and $K_i^{\min}$ denote the element-wise maximum and minimum of the key states within block i. Next, we compute a score s_i for each block based on the input query states. During standard decoding, only a single token is generated at each step, and thus only one query state is produced. However, in the verification phase, the model receives a sequence of candidate tokens generated by the draft module, resulting in multiple query states. To accommodate this situation, we first compute per-query scores for each block and then aggregate them into a single importance score.

$$s_{i,j} = \max\left(q_j \left(K_i^{\max}\right)^\top,\ q_j \left(K_i^{\min}\right)^\top\right) \tag{2}$$

$$s_i = f(s_{i,1}, s_{i,2}, \ldots, s_{i,M}) \tag{3}$$

where $s_{i,j}$ denotes the score between the j-th query state and the key state of block i, and f is a reduction function (e.g., max, mean, or last), with *last* referring to the query state of the most recently verified token.

Additionally, we retain a small number of recent *local tokens*, maintained as a fixed-size window. The last component is a buffer. It keeps tokens that have been partially verified and are awaiting correction. During forward phase, the key states of all candidate tokens are temporarily kept in this buffer so that invalid tokens can be removed once verification is completed. Although we describe these four components separately, they are contiguous in token order and are handled as a single continuous KV segment during the attention forward pass.

3.3 Rectified with Full Verification

During decoding, the errors introduced by partial verification gradually accumulate, causing the generated output to drift over time. In practice, this accumulation comes from two sources: the key–value

states of partially verified tokens are inaccurate, and the retrieved context changes as generation progresses. A practical solution is to periodically apply full verification to eliminate these accumulated errors.

Before entering the verification forward pass, the partially verified tokens are prepended to the newly generated candidate tokens. During verification, we perform the attention computation using the full KV cache and refresh the partial KV cache accordingly. In this forward pass, we compute key–value states for both the partially verified tokens and the candidate tokens. Afterward, we update the retrieval tokens and the local window tokens. Finally, once the candidate tokens have been evaluated, we remove the invalid tokens from both the partial and the full KV cache.

We illustrate the three verification processes in Figure 2. When the sequence length is smaller than the partial KV cache budget, partial verification is disabled. As the sequence grows and exceeds the partial budget, we perform a full verification once to initialize the partial KV cache, then smoothly transition into partial verification. Additionally, we set a maximum buffer size for the partial KV cache. When the total number of partially verified and candidate tokens exceeds this buffer, we switch back to full verification and refresh the partial KV cache. By adjusting the buffer size, we can control how frequently full verification is triggered.

Furthermore, when memory is constrained, a common strategy for long-context inference is KV cache offloading. Since the partial and draft cache are significantly smaller than the full KV cache, we offload only the full KV cache to host memory. At each layer, we keep only its key and value states on device and prefetch the next layer when needed. Under this setting, partial verification is especially helpful because it reduces full cache access and lowers PCIe transfer overhead, thus improving inference efficiency. Overall, the framework of SpecPV is summarized in Appendix A.

4 Experiments

In this section, we design a series of experiments to evaluate SpecPV in both model performance and generation efficiency.

4.1 Setup

Models We evaluate SpecPV on LLaMA-3.1-8B-Instruct (Grattafiori et al., 2024) and the Qwen-3 series (4B, 8B and 14B) (Yang et al., 2025a). For the draft module, we adopt the EAGLE-3 framework (Li et al., 2025b), a state-of-the-art approach for self-speculative decoding. The publicly released EAGLE-3 models are trained with a 2K context window. Building on these released weights, we further apply YARN-based long-context adaptation (Peng et al., 2023) using 6,400 PG-19 samples at a 32K sequence length. Additional details are provided in Appendix B.

Tasks For efficiency evaluation, we construct a story continuation task on PG-19 (Rae et al., 2019), which allows us to measure speedup under different context lengths. For quality evaluation, we use tasks from LongBench v1 (Bai et al., 2024a) and v2 (Bai et al., 2024b), covering two categories: document summarization and question answering.

Baselines We compare the efficiency of SpecPV against three speculative decoding baselines:

- **TriForce:** TriForce (Sun et al., 2024) employs an independent draft model and performs hierarchical speculation for long-context generation. The original method supports only LLaMA-2 models, and pretraining a standalone draft model from scratch would require substantial resources. Therefore, for LLaMA-3, we use Qwama-0.5B-Instruct (Turboderp, 2024) as the draft model, which is based on Qwen2-0.5B-Instruct but adapted to the LLaMA-3 vocabulary.
- **TokenSwift:** TokenSwift (Wu et al., 2025) adopts a Medusa-style draft head and uses a partial KV cache for drafting. It primarily focuses on ultra–long sequence generation, and therefore incorporates additional techniques such as token reutilization and contextual penalty.
- **EAGLE3-YARN:** We directly use our YARN-adapted EAGLE-3 models for speculative decoding generation.

Metrics We evaluate both the generation quality and the efficiency of SpecPV. For efficiency, we report two metrics:

- **Overall Speedup α:** The decoding time of all methods is compared against the standard autoregressive baseline. Speedup is computed as:

$$\text{Speedup} = \frac{\text{Throughput}_{\text{SpecPV}}}{\text{Throughput}_{\text{AR}}} \tag{4}$$

Table 1: Experimental results of LLaMA3.1-8B-Instruct across different context length. We compare SpecPV with various baselines using partial KV caches of 8K, 4K, and 2K. In the table, α denotes the micro-averaged throughput speedup, while τ represents the macro-averaged draft accept length.

Method	10K		20K		30K		40K		50K		60K	
	α	τ	α	τ	α	τ	α	τ	α	τ	α	τ
TriForce	1.72×	2.62	2.44×	2.82	2.65×	2.61	3.10×	2.80	3.42×	2.82	3.17×	2.61
TokenSwift	1.41×	1.64	1.70×	1.73	1.83×	1.73	1.98×	1.77	2.06×	1.77	2.11×	1.75
EAGLE3-YaRN	2.49×	3.38	2.74×	3.49	2.82×	3.38	3.02×	3.57	3.06×	3.46	3.11×	3.48
SpecPV-8K	2.53×	3.34	3.54×	3.51	4.23×	3.42	4.88×	3.56	5.31×	3.46	5.80×	3.57
SpecPV-4K	2.92×	3.29	4.00×	3.57	4.57×	3.43	5.46×	3.67	5.66×	3.52	6.15×	3.56
SpecPV-2K	2.88×	3.30	3.83×	3.42	4.75×	3.46	5.21×	3.55	5.79×	3.56	6.29×	3.57

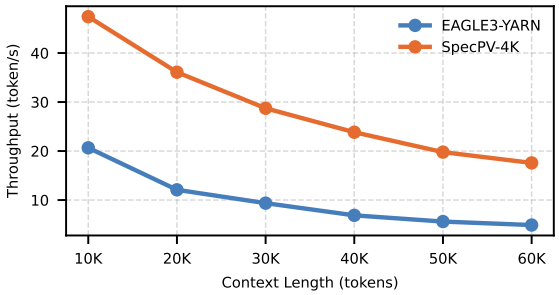

Figure 4: Decoding throughput of LLaMA3.1-8B-Instruct on a single RTX 4090 GPU with KV cache offloading.

- **Draft Accept Length τ:** We measure the average number of tokens accepted per verification step. Our metric allows an accept length of 0 when all drafted tokens are rejected.

For quality evaluation, we adopt task-appropriate metrics:

- **Question Answering:** We measure exact-match accuracy, comparing the generated answer with the ground truth after normalization.
- **Summarization:** We measure ROUGE-L (Lin, 2004), which measures longest common subsequence overlap, and BLEURT (Sellam et al., 2020), a learned metric that evaluates semantic similarity between sentences.

Implementations For consistency across baselines, we implement all attention operations using the scaled dot product attention of Pytorch. We evaluate decoding efficiency on an A100 80GB GPU with context lengths up to 60K. In memory-constrained settings, we test efficiency on a single RTX 4090 24GB GPU with KV cache offloading.

4.2 Generation Efficiency

We construct a test dataset from PG-19 by extracting segments of varying lengths while ensuring that sentences remain as complete as possible. Each model is then prompted to continue the story, generating up to 1024 tokens with a temperature of 0. For SpecPV generation, we use mean reduction for KV cache retrieval and we size the buffer to accommodate all tokens processed in a single verification step with an additional margin of 20. We only modify the verification process and keep the drafting process unchanged.

Table 2: Summarization performance on GovReport and QMSum (temperature = 0) on Qwen3-8B and LLaMA3.1-8B-Instruct. RL denotes ROUGE-L and BLT denotes BLEURT.

Models	Budget	GovReport		QMSum	
		RL	BLT	RL	BLT
LLaMA3.1	Full	35.3	42.7	25.0	44.9
	4096	34.4	42.6	25.0	44.1
	2048	33.8	42.5	24.9	44.2
	512	30.7	42.3	24.3	44.2
Qwen3	Full	33.7	41.4	23.9	42.4
	4096	33.3	41.3	23.6	42.0
	2048	33.1	41.3	23.8	41.9
	512	31.1	41.3	22.8	41.2

We compare SpecPV with other baselines on LLaMA3.1-8B-Instruct across different context length. The results are shown in Table 1. The partial KV budget for both TriForce and TokenSwift is set to 4K. For TokenSwift, its design primarily targets ultra-long sequence generation, typically beyond 20K tokens. Under our evaluation settings, techniques such as token reutilization have limited impact, resulting in relatively modest speedup. For TriForce, the hierarchical verification design yields solid acceleration in long-context settings. However, it does not refresh the partial KV cache, and each verification step relies on full verification to remain lossless. In addition, its standalone draft model yields slightly shorter draft accepted length in our experiments. These factors together

Table 3: Experimental results on the Qwen3 series across different context length. We report the throughput speedup (α) and draft accept length (τ) under various partial KV cache budgets. For the 60K context length, we set the YARN scaling factor of Qwen3 to 2.0 to accommodate the extended context window.

Size	Method	10K		20K		30K		40K		50K		60K	
		α	τ	α	τ	α	τ	α	τ	α	τ	α	τ
4B	EAGLE3-YaRN	2.43×	3.27	2.59×	3.28	2.61×	3.21	2.54×	3.05	2.70×	3.13	2.79×	3.18
	SpecPV-8K	2.46×	3.35	3.50×	3.30	4.02×	3.23	4.50×	3.14	5.04×	3.11	5.52×	3.11
	SpecPV-4K	2.55×	3.37	3.40×	3.11	4.22×	3.17	4.73×	3.17	5.44×	3.28	5.89×	3.29
	SpecPV-2K	2.76×	3.39	3.69×	3.31	4.31×	3.25	4.84×	3.20	5.32×	3.15	5.82×	3.14
8B	EAGLE3-YaRN	2.44×	3.23	2.65×	3.25	2.79×	3.35	2.78×	3.35	2.78×	3.29	2.83×	3.22
	SpecPV-8K	2.39×	3.02	3.33×	3.19	4.00×	3.19	4.46×	3.25	4.97×	3.27	5.31×	3.11
	SpecPV-4K	2.44×	2.98	3.47×	3.06	4.13×	3.02	4.83×	3.37	5.21×	3.29	5.44×	3.06
	SpecPV-2K	2.53×	2.90	3.58×	3.05	4.14×	3.01	4.61×	3.13	5.24×	3.32	5.35×	3.01
14B	EAGLE3-YaRN	2.46×	3.03	2.74×	3.30	2.83×	3.34	3.12×	3.81	3.18×	3.78	2.98×	3.31
	SpecPV-8K	2.46×	2.91	3.35×	3.11	3.83×	2.98	5.06×	3.79	5.45×	3.57	5.63×	3.44
	SpecPV-4K	2.75×	2.89	3.68×	3.14	4.13×	3.02	5.37×	3.71	6.02×	3.69	5.83×	3.23
	SpecPV-2K	2.74×	2.79	3.73×	3.08	4.37×	3.10	5.37×	3.66	5.99×	3.60	5.73×	2.96

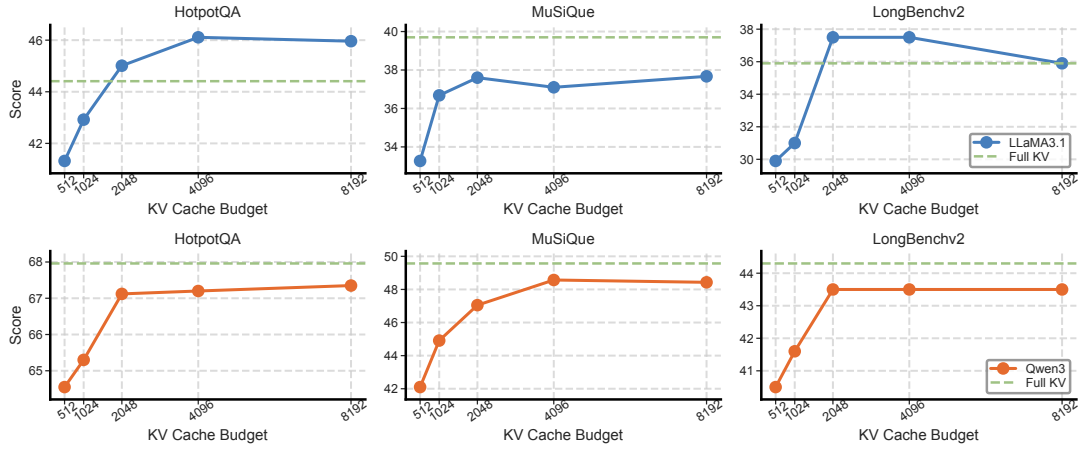

Figure 5: Accuracy on QA tasks under different partial KV cache budgets. For LongBench v2, samples exceeding 64K context length are excluded. For all datasets, we first generate a chain of thought and then prompt the model to produce a standardized final answer. For most datasets, SpecPV achieves performance comparable to full verification under a 4096 token KV budget.

contribute to its slower decoding speed compared with SpecPV. After applying our YARN-based long-context adaptation, the EAGLE-3 model also achieves notable draft accept length in long-context scenarios. When the context is relatively short, the speedup from partial verification is limited. However, as the context further increases, the benefits become far more significant. We also observe that different partial KV budgets lead to variations in draft accept length. In practice, the partial KV budget should be chosen flexibly based on the actual context length in order to balance efficiency and accuracy. Furthermore, we also evaluate performance on a single RTX 4090 GPU with KV cache offloading, shown in Figure 4. Since the primary bottleneck in this setting is PCIe transfer, and the partial KV cache is small enough to avoid host and device transfers, SpecPV achieves significant speedup.

We also evaluate SpecPV under different partial KV cache budgets on the Qwen3 series. For continuation tasks, we use the no-think mode. The results are shown in Table 3. Due to its mixed think and no-think output modes, Qwen3 produces slightly shorter draft accept length than LLaMA3.1. Overall, SpecPV provides consistent acceleration across different LLMs and model size.

4.3 Generation Quality

We evaluate generation quality on both summarization and question answering tasks. For summarization, we use the GovReport (Huang et al., 2021)

and QMSum (Zhong et al., 2021) datasets from LongBenchv1. Following standard practice, we measure the similarity between model-generated summaries and the human-provided ground truth using ROUGE-L and BLEURT. The results are shown in Table 2. Compared to full decoding, SpecPV achieves comparable end-task performance across different cache budgets, with only minor performance degradation. These results indicate that SpecPV largely preserves the semantic quality of the generated summaries while enabling more efficient inference.

For question answering, we use HotpotQA (Yang et al., 2018) and MuSiQue (Trivedi et al., 2022) from LongBenchv1 together with LongBenchv2 samples after excluding those that exceed a 64K context length. We first generate a chain of thought (CoT) and then prompt the model to produce a standardized final answer. For Qwen3, we generate the CoT in think mode to obtain stronger performance. The results are shown in Figure 5. Overall, as the KV cache budget decreases, model accuracy gradually declines. For most datasets, SpecPV achieves performance comparable to full verification with a 4096 token KV budget. In addition, we hypothesize that discarding part of the KV cache can sometimes reduce long-context noise, which explains why SpecPV even outperforms full verification on certain datasets. Overall, SpecPV maintains generation quality that is highly consistent with full verification.

Additionally, we conduct ablation studies on SpecPV hyperparameters, including buffer size and reduction type, in Appendix C. And provide detailed generation settings in Appendix D.

4.4 Case Study

Figure 6 presents a case study on Qwen3-8B for book summarization. We compare the output generated by full verification and SpecPV. From this example, we observe that although SpecPV uses partial KV verification, it preserves most of the key information, and the crucial factual content (highlighted in red) remains correct. However, the output generated with full verification includes more fine-grained details, shown in blue. Because SpecPV relies on only a subset of the KV cache, we conjecture that certain fine-grained details may be lost, which is not well reflected by standard accuracy metrics. We hope future long-context generation methods will better address this limitation.

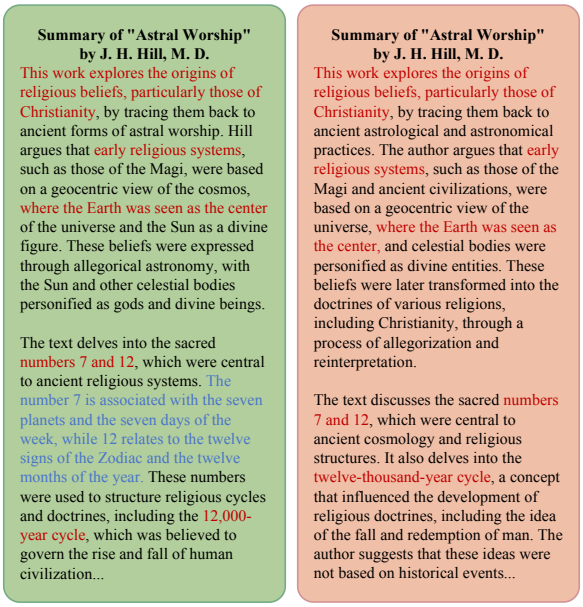

Figure 6: A case study on Qwen3-8B, where the LLM summarizes a 50K tokens document using full verification and SpecPV.

5 Conclusion

In this work, we propose SpecPV, a self-speculative decoding approach that performs fast verification using partial KV cache and periodically applies full verification to eliminate accumulated errors. By accelerating the verification stage within speculative decoding, SpecPV further improves generation efficiency in long-context settings. Our experiments show that SpecPV achieves up to 6× decoding speedup compared with standard autoregressive decoding, with only minor accuracy degradation. We believe this work contributes to enabling more efficient and practical speculative decoding for long-context LLM applications.

Limitations

While SpecPV demonstrates strong efficiency and quality trade-offs, several aspects merit further study. Compared to standard lossless speculative decoding methods, SpecPV introduces a controlled approximation via partial KV verification, as shown in Figure 6, it preserves key semantic and factual content but may occasionally miss fine-grained details, which are not always reflected by standard metrics. In addition, its acceleration performance depends on the draft model (here, an EAGLE3-style self-speculative decoding), and may vary across languages or application scenarios. Finally, recent advances in pre-training attention

architectures further improve native long-context modeling; exploring how speculative decoding can be combined with such approaches is a promising direction for future work.

References

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024a. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137, Bangkok, Thailand. Association for Computational Linguistics.

Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024b. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. *arXiv preprint arXiv:2412.15204*.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning*, pages 5209–5235. PMLR.

Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. 2023. Enhancing chat language models by scaling high-quality instructional conversations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3029–3051.

Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, and 1 others. 2024. Layerskip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12622–12642.

Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. 2024. Better & faster large language models via multi-token prediction. In *Proceedings of the 41st International Conference on Machine Learning*, pages 15706–15734.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun S Shao, Kurt Keutzer, and Amir Gholami. 2024. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *Advances in Neural Information Processing Systems*, 37:1270–1303.

Luyang Huang, Shuyang Cao, Nikolaus Parulian, Heng Ji, and Lu Wang. 2021. Efficient attentions for long document summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1419–1436.

Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, and 1 others. 2024. Openai o1 system card. *arXiv preprint arXiv:2412.16720*.

Kimi Team. 2025. Kimi linear: An expressive, efficient attention architecture. *arXiv preprint arXiv:2510.26692*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR.

Shenggui Li, Yikai Zhu, Chao Wang, Fan Yin, Shuai Shi, Yubo Wang, Yi Zhang, Yingyi Huang, Haoshuai Zheng, and Yineng Zhang. 2025a. Specforge: Train speculative decoding models effortlessly. https://github.com/sgl-project/specforge.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024a. Eagle-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 7421–7432.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024b. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning*.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025b. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*.

Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pages 74–81.

Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, and 1 others. 2024a. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*.

Fangcheng Liu, Yehui Tang, Zhenhua Liu, Yunsheng Ni, Duyu Tang, Kai Han, and Yunhe Wang. 2024b. Kangaroo: Lossless self-speculative decoding for accelerating llms via double early exiting. *Advances in Neural Information Processing Systems*, 37:11946–11965.

Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024c. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. In *International Conference on Machine Learning*, pages 32332–32344. PMLR.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, and 1 others. 2024. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 932–949.

Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. 2023. Yarn: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071*.

Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, Chloe Hillier, and Timothy P Lillicrap. 2019. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*.

Ranajoy Sadhukhan, Jian Chen, Zhuoming Chen, Vashisth Tiwari, Ruihang Lai, Jinyuan Shi, Ian En-Hsu Yen, Avner May, Tianqi Chen, and Beidi Chen. 2025. Magicdec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding. In *The Thirteenth International Conference on Learning Representations*.

Thibault Sellam, Dipanjan Das, and Ankur P Parikh. 2020. Bleurt: Learning robust metrics for text generation. In *Proceedings of ACL*.

Hanshi Sun, Zhuoming Chen, Xinyu Yang, Yuandong Tian, and Beidi Chen. 2024. Triforce: Lossless acceleration of long sequence generation with hierarchical speculative decoding. In *First Conference on Language Modeling*.

Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. Quest: Query-aware sparsity for efficient long-context llm inference. In *International Conference on Machine Learning*, pages 47901–47911. PMLR.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. Musique: Multi-hop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554.

Turboderp. 2024. Qwama-0.5b-instruct. `https://huggingface.co/turboderp/Qwama-0.5B-Instruct`. Accessed: 2025-11-15.

Haoran Wei, Yaofeng Sun, and Yukun Li. 2025. Deepseek-ocr: Contexts optical compression. *arXiv preprint arXiv:2510.18234*.

Tong Wu, Junzhe Shen, Zixia Jia, Yuxuan Wang, and Zilong Zheng. 2025. Tokenswift: Lossless acceleration of ultra long sequence generation. In *Forty-Second International Conference on Machine Learning*.

Heming Xia, Yongqi Li, Jun Zhang, Cunxiao Du, and Wenjie Li. 2025. SWIFT: On-the-fly self-speculative decoding for LLM inference acceleration. In *The Thirteenth International Conference on Learning Representations*.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*.

Ruyi Xu, Guangxuan Xiao, Haofeng Huang, Junxian Guo, and Song Han. 2025. Xattention: Block sparse attention with antidiagonal scoring. In *Forty-second International Conference on Machine Learning*.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025a. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Shang Yang, Junxian Guo, Haotian Tang, Qinghao Hu, Guangxuan Xiao, Jiaming Tang, Yujun Lin, Zhijian Liu, Yao Lu, and Song Han. 2025b. Lserve: Efficient long-sequence llm serving with unified sparse attention. *arXiv preprint arXiv:2502.14866*.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2024. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2369–2380.

Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. 2024. Draft& verify: Lossless large language model acceleration via self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11263–11282.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, and 1 others. 2023. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710.

Ming Zhong, Da Yin, Tao Yu, Ahmad Zaidi, Mutethia Mutuma, Rahul Jha, Ahmed Hassan, Asli Celikyilmaz, Yang Liu, Xipeng Qiu, and 1 others. 2021. Qmsum: A new benchmark for query-based multi-domain meeting summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5905–5921.

A SpecPV Algorithm Description

We provide the pseudocode of SpecPV in Algorithm 1 for clarity.

B YARN-Based Extension of the EAGLE-3 Module

For the original EAGLE-3 draft module, directly applying it to long-context scenarios fails to produce effective draft predictions. The core issue is that the decoder layer inside EAGLE-3 relies on RoPE position encoding, which does not extrapolate well to extended context lengths. Moreover, most publicly available EAGLE-3 models are trained on self-generated conversational data (Ding et al., 2023) with context windows around 2K tokens. As a result, we first need to extend its positional embeddings.

We extend the context length using YARN. Since YARN finetuning only repairs the positional embedding and does not require injecting additional knowledge, it can be trained with a relatively small amount of data. We construct a dataset of 6,400 samples from PG-19 with a 32K context window. We fine-tune the EAGLE-3 module based on SpecForge (Li et al., 2025a), and our training hyperparameters are listed in Table 4.

After training, we set the scaling factor to 32.0, which allows the model to extrapolate to a 64K context length directly. To enable the EAGLE-3 module to perform autoregressive generation more effectively, the authors adopt the training-time test technique. During training, the model performs multiple rounds of generation, allowing the loss to be aggregated across multiple decoding steps:

$$L = L_0 + \beta L_1 + \beta^2 L_2 + \cdots + \beta^n L_n \quad (5)$$

Following common practice, we set the discount factor β to 0.8. The training loss curves are shown in Figure 7. As the training-time test step increases, the loss consistently decreases.

Algorithm 1 SpecPV Generation

Input: target model $\mathcal{M}_{\text{tgt}}$, draft model $\mathcal{M}_{\text{draft}}$, prompt p, cache configuration $\mathcal{C}$, sampling configuration $\mathcal{S}$, max length $\mathcal{T}$, max new tokens $\mathcal{N}$
Output: generated sequence y
$\text{KV}_{\text{full}}, \text{KV}_{\text{partial}}, \text{KV}_{\text{draft}} \leftarrow \text{InitKV}(\mathcal{C})$
$h, x \leftarrow \text{ChunkPrefill}(\mathcal{M}_{\text{tgt}}, KV_{\text{full}}, \mathcal{M}_{\text{draft}}, KV_{\text{draft}}, p)$
$y \leftarrow \text{concat}(p, [x])$
for $t = 1$ **to** $\mathcal{T}$ **do**
  $\text{candidates} \leftarrow \text{TreeDraft}(\mathcal{M}_{\text{draft}}, \text{KV}_{\text{draft}}, h, x)$
  $\text{mode} \leftarrow \text{SelectMode}(\text{len}(y),\ KV_{\text{partial}})$
  if mode $=$ FULL **then**
    $KV_{\text{in}} \leftarrow KV_{\text{full}}$
  else if mode $=$ PARTIAL **then**
    $KV_{\text{in}} \leftarrow KV_{\text{partial}}$
  else if mode $=$ REFRESH **then**
    $KV_{\text{in}} \leftarrow \{KV_{\text{partial}},\ KV_{\text{full}}\}$
    $\text{candidates} \leftarrow \text{concat}(x_{\text{partial}}, \text{candidates})$
  end if
  $\text{logits},\ h \leftarrow \text{TreeDecode}(\mathcal{M}_{\text{tgt}},\ \text{candidates},\ KV_{\text{in}})$
  $x \leftarrow \text{PostEvaluate}(\text{candidates}, \text{logits}, \text{KV}_{\text{in}}, \mathcal{S})$
  Append x to y
  if $\text{len}(y) - \text{len}(p) \geq \mathcal{N}$ **or** EOS $\in x$ **then**
    break
  end if
end for

Table 4: Training hyperparameters for fine-tuning the EAGLE-3 module using SpecForge.

Hyperparameter	Value
Num epochs	1
Learning rate	2e-5
Warmup ratio	0.05
Max length	32768
Training-TimeTest length	4
Tensor parallel size (TP)	1
Data parallel size (DP)	4
Draft micro batch size	1
Draft global batch size	16
Draft accumulation steps	4
Scaling factor	16.0

C Ablation Study

In this appendix, we focus on the impact of different query-reduction strategies and the influence of buffer size.

We evaluate different reduction strategies on Qwen3-8B and LLaMA-3.1-8B-Instruct using the GovReport dataset under a 2K KV budget. To more precisely characterize the effects of different reduction types and refresh intervals, we directly compare the model-generated summarization outputs with those obtained under full verification. The results are shown in Table 5. Overall, the choice of reduction type has only a minor effect on perfor-

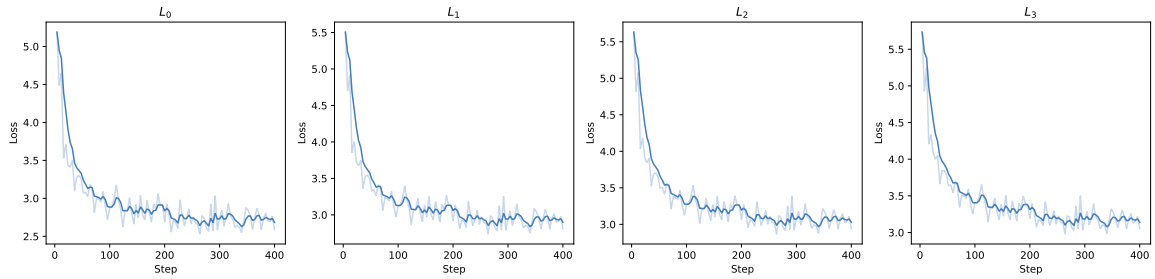

Figure 7: Loss curves of EAGLE3 YARN fine-tuning on LLaMA3.1-8B-Instruct. The plots show both the raw loss and its time-weighted exponential moving average (EMA) across training steps.

Table 5: Different reduction results on GovReport. Mean, Max, and Last denote different retrieval score aggregation strategies.

Models	Mean		Max		Last	
	RL	τ	RL	τ	RL	τ
LLaMA3.1	56.1	3.53	55.8	3.52	54.0	3.47
Qwen3	56.0	2.77	55.5	2.76	55.4	2.77

mance. Mean reduction yields the highest similarity and also produces slightly longer draft accept length. Consequently, we adopt mean reduction in all other experiments.

By adjusting the buffer size, we effectively control the interval for partial KV cache refreshes. We evaluate this effect on QMSum, as shown in Figure 8. Across different models, we observe a similar trend: as the refresh interval increases, the similarity between SpecPV and full verification gradually decreases, indicating that periodically partial KV cache updates help preserve model performance. To balance efficiency and accuracy, we typically set the buffer size to the number of tokens involved in one verification step plus 20.

D Generation Settings

For the LLaMA 3.1-8B-Instruct model, which supports up to a 128K context length and operates only in a instant response mode, we set the generation temperature to 0 across all experiments.

For the Qwen3 series, which provides both thinking and non-thinking modes with officially recommended configurations, we adopt different settings depending on the task. In the thinking mode, we use temperature = 0.6, TopP = 0.95, and TopK = 20. In the non-thinking mode, we set temperature = 0. For summarization and continuation tasks, we directly use the non-thinking mode, while for question answering tasks, we use the thinking mode.

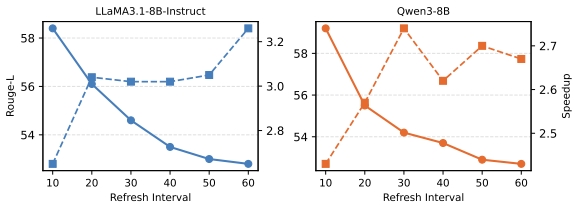

Figure 8: Experimental results of different refresh intervals on QMSum. Solid lines indicate ROUGE-L, and dashed lines indicate speedup. Overall, larger refresh intervals lead to greater deviation from full verification, but yield higher speedup.

Table 6: Summarization performance under different temperatures and partial KV budgets. RL denotes ROUGE-L and BLT denotes BLEURT.

Temperature	Budget	GovReport		QMSum	
		RL	BLT	RL	BLT
$t = 0.7$	Full	34.8	42.4	24.9	43.9
	4096	34.5	42.3	24.5	43.7
	2048	33.4	42.5	24.4	43.6
	512	31.7	42.1	23.9	43.0
$t = 1.0$	Full	34.5	42.2	24.2	44.0
	4096	33.9	42.1	23.9	43.3
	2048	32.9	42.4	24.2	43.1
	512	30.0	41.8	23.4	43.2

In addition, for the SpecPV algorithm, we set the block size to 16, the number of sink tokens to 32, and the local window size to 128 tokens. The buffer size is fixed at 20.

E Ablation on Temperature

For non-deterministic sampling, we evaluate the performance of LLaMA3.1-8B-Instruct under different temperature settings. We fix $top_k = 0$ and $top_p = 0.9$, following commonly adopted sampling configurations in LLM generation. Specifically, we conduct experiments with $T = 0.7$ and

Table 7: Experimental results under different context lengths and temperatures for LLaMA3.1-8B-Instruct. α denotes the throughput speedup over autoregressive decoding, and τ represents the average draft accept length.

Temp.	Method	10K		20K		30K		40K		50K		60K	
		α	τ	α	τ	α	τ	α	τ	α	τ	α	τ
$t = 0.7$	EAGLE3-YARN	2.22×	3.12	2.53×	3.18	2.65×	3.13	2.81×	3.26	2.90×	3.31	2.88×	3.13
	SpecPV-8K	2.31×	3.05	3.16×	3.12	3.95×	3.17	4.45×	3.19	4.98×	3.20	5.49×	3.30
	SpecPV-4K	2.66×	3.02	3.64×	3.14	4.16×	3.17	4.92×	3.29	5.49×	3.24	5.77×	3.24
$t = 1.0$	EAGLE3-YARN	2.20×	2.70	2.25×	2.61	2.42×	2.68	2.52×	2.75	2.65×	2.81	2.67×	2.81
	SpecPV-8K	2.06×	2.42	2.73×	2.46	3.50×	2.55	4.20×	2.70	4.65×	2.64	5.16×	2.81
	SpecPV-4K	2.45×	2.58	3.14×	2.66	3.96×	2.67	4.54×	2.76	5.18×	2.78	5.46×	2.85

$T = 1.0$, and analyze both the summarization quality and acceleration performance under non-deterministic decoding.

We observe that different temperature settings have relatively limited impact on the overall generation quality. SpecPV maintains comparable ROUGE-L and BLEURT scores to full verification under non-deterministic decoding, while a small KV budget (e.g., 512) leads to a more noticeable ROUGE-L drop due to the loss of long-context information, although the semantic quality remains largely preserved. In terms of acceleration, SpecPV shows consistent speedup trends across different temperatures. However, the average accept length becomes smaller when $T = 1.0$, since higher temperatures produce smoother token distributions and increase the discrepancy between the draft and target model distributions, resulting in more frequent draft-target mismatches during speculative decoding.